

Algorithms & Data Structures

Industry Scenario Problems – Practice Set (Days 1 to 5)

Module: Algorithms & Data Structures using Java · Batch: PGCP-AC August 2026

General Instructions

Dear Students,

This practice set is designed to take you beyond textbook exercises. Every problem is based on a situation you are likely to face in a real software job, and each one applies a data structure we have covered from Day 1 to Day 5. Interviewers today rarely ask “implement a stack”. They describe a business problem and expect you to recognise the right structure. This set prepares you for exactly that.

These problems are not meant to be finished in a few minutes. Read each scenario carefully, design your solution on paper first, and only then write code. Be prepared to answer the follow-up questions given with each problem, since these are the questions an interviewer is likely to ask next.

Students are expected to follow these steps for every problem:

1. **Understand the Scenario** – Restate the business problem in your own words. Identify the input, the output and the operations the system must support.
2. **Choose the Data Structure** – Name the structure you will use and justify your choice in 2–3 lines. Also mention one alternative you considered and why you rejected it.
3. **Approach / Algorithm** – Explain your approach step by step in simple English. Do not start with Java code.
4. **Dry Run** – Trace the given example by hand. Draw the array, stack, queue, list or tree after every operation.
5. **Pseudocode** – Write the logical steps of each operation.
6. **Java Implementation** – Write a complete Java program. You are required to build your own stack, queue, linked list and tree classes. Do not use `java.util` collections unless the problem specifically allows them.
7. **Complexity** – State the time and space complexity of every operation, with a one-line reason.
8. **Test** – Run the given example, every case listed under *Edge Cases to Consider*, and at least two inputs of your own.

Use of AI Tools

The purpose of this set is to build **your** problem-solving ability. You are therefore not permitted to paste a problem into an AI tool and ask it to write the code. Doing so defeats the purpose of the exercise, and you will not be able to explain the solution in an interview.

Where a problem has a **Check with AI** box, you may use an AI tool only **after** your own program is working. Paste your code and ask: “Do not rewrite my code. List inputs where it fails and explain why.” Verify every point it raises yourself, as AI tools are frequently wrong.

If you are stuck on a problem, first attempt it on your own for some time. Then read the Hint, and if you still need help, discuss the *idea* with your classmates. Please do not share code with each other.

Problem Map

#	Scenario	Core Concept	Day
1	Peak traffic window for a web server	Complexity, sliding window	1, 2
2	Brokerage app - best trade report	Single-pass array scan	1, 2
3	IoT telemetry buffer	Dynamic array grow / shrink	2
4	API rate limiter	Circular queue (ring buffer)	2
5	HTML / XML tag validator	Stack	2
6	Billing formula engine	Two stacks, infix evaluation	2
7	Round-robin CPU scheduler	Circular queue	2
8	Server CPU monitoring dashboard	Deque (monotonic)	2
9	Music playlist player	Circular doubly linked list	3
10	Banking big-number addition	Linked list + recursion	3, 4
11	Memory allocator free list	Sorted linked list, merging nodes	3
12	Disk usage analyzer	Recursion on an N-ary tree	4, 5
13	Hackathon team builder	Recursion, backtracking, pruning	4
14	E-commerce price filter	BST range query, skewed trees	5
15	Company org chart	General tree, BFS, common ancestor	5
16	Saving a tree to disk / network	Tree serialization	4, 5

Question 1 - Peak Traffic Window for a Web Server

Topic: Complexity analysis, arrays, sliding window · Day 1, 2 · Level: Intermediate

Scenario

An e-commerce company records the number of requests its server receives every minute. Before a festival sale, the infrastructure team wants to know the **busiest k-minute window** of the previous day, so that they can plan extra servers.

A junior developer wrote the code below. On one month of per-second data, it takes many minutes to finish.

```
long best = 0;
for (int i = 0; i + k <= n; i++) {
    long sum = 0;
    for (int j = i; j < i + k; j++) sum += req[j];
    best = Math.max(best, sum);
}
```

Example

Input: req = [120, 340, 560, 210, 90, 680, 720, 150], k = 3
Output: Peak 3-minute load = 1550 requests (minutes 6 to 8)

Conditions

- Your final solution must run in $O(n)$ time and $O(1)$ extra space.
- Report both the peak total and the window (start and end minute, 1-based).
- If two windows have the same total, report the earliest one.
- Use `long` for sums. Justify why `int` is risky here.

Your Tasks

1. State the time complexity of the existing code in terms of n and k .
2. Estimate the running time of the existing code and of your solution for $n = 2,592,000$ (one month of seconds) and $k = 3,600$ (one hour). Assume 10^8 simple operations per second. Show the arithmetic.
3. Design the $O(n)$ approach and dry-run it on the example, writing the window sum after every step.
4. Implement both versions. Add an operation counter and print the counts for $n = 10^3$, 10^5 and 10^6 with $k = 1000$.
5. Explain why your measured counts agree, or disagree, with your Big-O prediction.

Edge Cases to Consider

- $k = 1$, $k = n$, and $k > n$ (what should happen?)
- All values zero
- An empty array

Think Before Coding

When the window moves one step to the right, exactly one value enters and one value leaves. Do you really need to add up all k values again?

Interview Angle

“Now find the **shortest** window whose total is at least T requests.” Can the window still slide, or must it grow and shrink? What property of the data makes that possible?

On the Job

Monitoring tools such as Grafana dashboards, autoscaling rules and billing reports all compute rolling sums like this.

Question 2 - Brokerage App: Best Trade Report

Topic: Arrays, single-pass algorithms · Day 1, 2 · Level: Intermediate

Scenario

A stock brokerage app sends users a monthly report: “If you had bought once and sold once this month, the best profit you could have made was ...”. The closing prices for one stock are stored in an array, one value per trading day.

The report must show the profit, the buy day and the sell day. The app generates this report for 2 crore users, so an $O(n^2)$ solution is not acceptable.

Example

```
Input:  prices = [7, 1, 5, 3, 6, 4]
Output: Buy on day 2 at 1, sell on day 5 at 6, profit = 5

Input:  prices = [9, 7, 4, 3, 1]
Output: No profitable trade this month (profit = 0)
```

Conditions

- You must buy before you sell, on different days.
- Solve it in one pass, in $O(n)$ time and $O(1)$ space. Do not use sorting.
- Days are 1-based in the output.

Your Tasks

1. Write the brute-force solution and state its complexity.
2. Design the single-pass solution. In one sentence, say what single value you must remember as you scan.
3. Dry-run it on the first example, showing the value you remember and the best profit so far after each day.
4. Implement it, and return a small result object `Trade(buyDay, sellDay, profit)` instead of printing inside the method.
5. **Extension:** Allow *any number* of buy-sell pairs, never holding more than one share at a time. For the first example, the answer is 7 (buy at 1, sell at 5, then buy at 3, sell at 6). Implement this in $O(n)$ as well.

Edge Cases to Consider

- Prices that only fall
- All prices equal
- A single day
- The best buy day comes *after* a lower price that has no later high

Think Before Coding

As you walk through the days, the best sell for “today” depends on only one thing from the past. What is it?

Interview Angle

“What if the user can make at most **two** trades?” You don't need to code this. Explain in words why the simple greedy idea from the extension no longer works.

On the Job

Fintech reporting, ‘missed opportunity’ analytics, and any query of the form “best difference with order preserved”.

Question 3 - IoT Telemetry Buffer (Dynamic Array)

Topic: ArrayList internals, amortized analysis · Day 2 · Level: Advanced

Scenario

A smart-factory gateway receives temperature readings from sensors. Readings arrive in bursts: thousands in a minute during a fault, then almost nothing for hours. The device has very little RAM, so the buffer must **grow when needed and shrink when it is mostly empty**.

Write your own `TelemetryBuffer`, backed by a plain `double[]`, with these operations:

Operation	Meaning
<code>append(double r)</code>	Add a reading at the end; double the capacity if full
<code>removeLast()</code>	Remove and return the newest reading; shrink if the buffer is mostly empty
<code>get(int i)</code>	Reading at index <code>i</code>
<code>averageOfLast(int k)</code>	Average of the newest <code>k</code> readings
<code>size(), capacity()</code>	Current element count and array length

Example

```
Start:          capacity = 4 (minimum capacity is 4)
append 9 readings -> capacity grows 4 -> 8 -> 16, size = 9
removeLast x 6   -> size = 3, capacity = 8
                  (when size dropped to 4 = 16/4, capacity halved to 8)
```

Conditions

- Grow: when the array is full, double the capacity.
- Shrink: when size becomes **one quarter** of the capacity, halve the capacity. Never go below 4.
- Do not use `ArrayList` or `Arrays.copyOf`. Copy the elements yourself.
- Keep a counter of every element copied during resizing.

Your Tasks

1. Dry-run the example and draw the array after every resize.
2. Implement the class and print `size`, `capacity` and `copies` after each phase of the example.
3. Run 1,000,000 random append/remove operations and print `copies / operations`. What constant does it stay below?
4. Now change the shrink rule to “halve when size becomes **half** the capacity”. Find a sequence of operations that makes almost every operation $O(n)$. This problem is called *thrashing*. Show the copy count exploding.
5. Explain in 4-5 lines why the quarter rule avoids thrashing.

Edge Cases to Consider

- `removeLast` on an empty buffer
- `averageOfLast(k)` with $k > \text{size}$ or $k \leq 0$
- Repeated grow/shrink right at the boundary

Hint

Thrashing happens when one append triggers a grow and the very next remove triggers a shrink. Try building to exactly a power of two, then alternating append and remove.

Interview Angle

“Why does Java's `ArrayList` grow by $1.5\times$ and never shrink automatically? What does `trimToSize()` do?” Look these up in the JDK source and answer in your own words.

Check with AI (after you finish)

After your class works, ask an AI to suggest a sequence of operations that would break your resize logic. Run each suggestion yourself.

Question 4 - API Rate Limiter (Ring Buffer)

Topic: Circular queue · Day 2 · Level: Advanced

Scenario

A payment gateway exposes a public API. To protect it from abuse, each user may make **at most K requests in any window of W seconds**. A request that would exceed the limit is rejected with HTTP 429 (Too Many Requests).

Design a `RateLimiter` for one user, with `boolean allow(long timestampSec)`. It must use a **fixed-size circular array of K slots**, so memory never grows with traffic.

Example

K = 3 requests, W = 10 seconds (window for a request at time t is (t-10, t])

Time :	1	2	3	4	11	12	13	14
Result:	OK	OK	OK	429	OK	OK	OK	429

At t=4 : requests at 1, 2, 3 are all inside (-6, 4] -> 3 already, reject

At t=11: request at 1 is outside (1, 11]; only 2, 3 count -> allow

Conditions

- Timestamps arrive in non-decreasing order.
- Rejected requests are **not** stored.
- `allow()` must be O(1). Do not scan all K slots.

Your Tasks

1. Explain why only the *oldest* stored request ever needs to be checked.
2. Dry-run the example. After each call, draw the ring buffer with its head index.
3. Implement `RateLimiter` with your own circular-array logic, without `java.util` queues.
4. Extend it to many users: `allow(String userId, long t)`. A `HashMap<String, RateLimiter>` is allowed here. If 1 crore users each have K = 100, what is the memory cost? Is it acceptable?
5. Write a test that sends 1,000 requests at the same second and checks that exactly K are allowed.

Edge Cases to Consider

- K = 1
- Many requests with the same timestamp
- A request exactly W seconds after the oldest one (is it inside or outside the window?)
- A long gap with no traffic, then a burst

Think Before Coding

The buffer holds the last K *accepted* requests. If the oldest of them is still inside the window, how many requests are inside the window?

Interview Angle

"This is the sliding-log algorithm. Compare it with a **fixed-window counter**, which resets every W seconds. Show a sequence where the fixed window lets through 2K requests in a short burst."

On the Job

API gateways (Nginx, AWS API Gateway), login-attempt throttling, and OTP request limits all use this idea.

Question 5 - HTML / XML Tag Validator

Topic: Stack · Day 2 · Level: Intermediate

Scenario

A content-management system lets editors paste HTML into a blog post. Broken markup breaks the whole page layout. Before saving, the system must validate that every tag is properly closed and nested. If not, it must report the **line number and the exact problem**.

Example

```
Input (one tag or text per line):
1: <html>
2: <body>
3: <div class="post"><p>Hello</p>
4: <br/>
5: </body>
6: </html>

Output:
Line 5: expected </div> but found </body>
```

Conditions

- Tag names are case-insensitive: `<DIV>` matches `</div>`.
- Ignore attributes: `<div class="post">` is a `div` tag.
- Self-closing tags such as `<br/>` and `<img .../>` need no closing tag.
- Use your own stack. Do not use regex libraries for the matching itself (`String` methods are fine).
- Report only the **first** error.

Your Tasks

1. Identify every kind of error that can occur. There are at least three. Give one input for each.
2. Decide what the stack must store so that you can report useful line numbers.
3. Write a small tokenizer that extracts tags from a line (a line may hold several tags).
4. Implement the validator and test it on the example and on one error input of each kind.
5. State the time and space complexity in terms of the number of tags.

Edge Cases to Consider

- A closing tag when nothing is open
- A file that ends with tags still open (report the line where each was opened)
- Text containing `<` that is not a tag, for example `5 < 7`
- Empty input

Hint

Push the tag *name* and its line number together, for example as a small `OpenTag` object.

Interview Angle

“Browsers are forgiving: they auto-close a missing `</p>`. How would you add a rule that auto-closes a `p` tag instead of reporting an error?”

On the Job

Linters, IDE error highlighting, JSON/XML parsers and compilers all use a stack to check nesting.

Question 6 - Billing Formula Engine

Topic: Stacks, infix expression evaluation · Day 2 · Level: Advanced

Scenario

A SaaS billing product lets each business customer define its own pricing formula. The finance team types the formula as text, and the system evaluates it for every invoice using that order's values.

Example

```
Formula:  price * qty - discount + (price * qty * gst) / 100
Variables: price = 250, qty = 4, discount = 100, gst = 18

Output:   1080
          (1000 - 100 + 18000 / 100 = 1000 - 100 + 180)
```

Conditions

- Support `+` `-` `*` `/`, parentheses, integer constants and variable names (letters only).
- Use integer arithmetic (`long`). `*` and `/` bind tighter than `+` and `-`. All operators are left-associative.
- Evaluate in a **single left-to-right pass** using two of your own stacks: one for values, one for operators.
- Variables come from a `Map<String, Long>`, which is allowed here.
- Spaces may or may not be present: `price*qty` is valid.

Your Tasks

1. Write the operator precedence table, then dry-run the example. Show both stacks after every token.
2. Implement a tokenizer that handles multi-digit numbers and multi-letter variables.
3. Implement the evaluator.
4. Produce clear error messages for each kind of invalid input (see below), instead of a stack trace.
5. State the time and space complexity in terms of the formula length.

Edge Cases to Consider

- Division by zero (for example, when `qty = 0` appears in a denominator)
- An unknown variable name
- Mismatched parentheses: `(a + b` or `a + b)`
- Two operators in a row: `a + * b`
- **Challenge:** unary minus, for example `-discount + price`

Think Before Coding

When you read an operator, which operators already on the stack must be applied first? Write the rule as one sentence before coding.

Interview Angle

“Evaluating the same formula a million times re-parses it a million times. How could you parse it **once**, convert it to postfix, and evaluate only the postfix form per invoice?”

Check with AI (after you finish)

Ask an AI for 10 tricky formulas that might break your evaluator, along with their expected results. Check each expected result by hand before you trust it.

Question 7 - Round-Robin CPU Scheduler

Topic: Circular queue, simulation · Day 2 · Level: Advanced

Scenario

An operating system shares one CPU among many processes using **round-robin scheduling**. Each process runs for at most a fixed time slice (the *quantum*). If it hasn't finished, it goes to the back of the ready queue. Build a simulator that prints the execution timeline and the waiting time of each process.

Example

Quantum = 2. All processes arrive at time 0, in order P1, P2, P3.
Burst times: P1 = 5, P2 = 3, P3 = 1

Timeline: | P1 0-2 | P2 2-4 | P3 4-5 | P1 5-7 | P2 7-8 | P1 8-9 |

Process	Burst	Completion	Turnaround	Waiting
P1	5	9	9	4
P2	3	8	8	5
P3	1	5	5	4

Average waiting time = 4.33

Conditions

- Use your own circular queue of `Process` objects (id, burst, remaining, arrival).
- Turnaround = completion – arrival. Waiting = turnaround – burst.
- Merge consecutive slices of the same process in the timeline.

Your Tasks

1. Dry-run the example by hand and draw the queue contents at every time step.
2. Implement the simulator for processes that all arrive at time 0.
3. **Extension:** Support different arrival times, for example P1@0, P2@1, P3@4. Decide (and document) the order when a new arrival and a pre-empted process join the queue at the same instant.
4. Run your simulator with quantum = 1, 2, 4 and 100 on the same input. Explain what happens to the average waiting time and to the number of context switches.
5. When the quantum is very large, which other scheduling algorithm does round robin behave like?

Edge Cases to Consider

- A burst exactly equal to the quantum
- A single process
- The CPU sits idle because no process has arrived yet
- A burst time of 0

Hint

Simulate with a `currentTime` variable. Before re-queuing a process that just ran, first enqueue every process that arrived during that slice.

Interview Angle

“Why don't modern operating systems use a very small quantum, like 1 microsecond?” Mention context-switch cost.

On the Job

OS schedulers, thread pools, load balancers that distribute requests round-robin, and game loops.

Question 8 - Server CPU Monitoring Dashboard

Topic: Deque, monotonic queue · Day 2 · Level: Advanced

Scenario

A monitoring dashboard receives a server's CPU usage once per second. Its alert panel shows the **maximum CPU usage over the last k seconds**, updated every second. With thousands of servers and $k = 300$, recomputing the maximum from scratch every second is too slow.

Example

Input: `cpu = [10, 20, 15, 30, 25, 5, 40]`, `k = 3`
Output: `[20, 30, 30, 30, 40]`

Windows: `[10 20 15] [20 15 30] [15 30 25] [30 25 5] [25 5 40]`

Conditions

- Final solution: $O(n)$ total time, $O(k)$ extra space.
- Implement your own deque (array-based circular or doubly linked). Do not use `ArrayDeque`.
- Store **indices** in the deque, not values.

Your Tasks

1. Write the $O(n \cdot k)$ brute force first and keep it as a correctness checker.
2. Design the deque approach. Write down the rule for when to remove from the **back** and when to remove from the **front**.
3. Dry-run the example, showing the deque contents (as indices and as values) after each second.
4. Implement it, then compare it against the brute force on 1,000 random arrays.
5. Explain why it is $O(n)$ even though there is a `while` loop inside the `for` loop.

Edge Cases to Consider

- $k = 1$ and $k = n$
- A strictly decreasing input
- All values equal (decide whether to keep duplicates in the deque)

Think Before Coding

If a newer reading is higher than an older one, can the older reading ever be the maximum of a future window?

Interview Angle

“Now show the **minimum** and the maximum together.” What changes? Can you reuse your class?

On the Job

Real-time monitoring (Datadog, Prometheus), trading systems with a rolling high and low, and network jitter tracking.

Question 9 - Music Playlist Player

Topic: Circular doubly linked list · Day 3 · Level: Intermediate

Scenario

You are building the playlist engine for a music-streaming app. Users skip forward and backward, use *Play Next* to queue a song immediately after the current one, remove songs, and turn *Repeat All* on and off. Every one of these actions must be $O(1)$, even for a 10,000-song playlist.

Operation	Behaviour
<code>add(song)</code>	Append to the end of the playlist
<code>next()</code> / <code>prev()</code>	Move the current song; with Repeat off, stop at either end
<code>playNext(song)</code>	Insert right after the current song
<code>removeCurrent()</code>	Remove the playing song; the next song starts playing
<code>setRepeat(boolean)</code>	Repeat on means the list wraps around
<code>moveTo(song, position)</code>	Reorder by drag and drop ($O(n)$ is fine here)

Example

<code>add A, B, C, D</code>	playlist: A B C D	current: A
<code>next()</code>		current: B
<code>playNext(X)</code>	playlist: A B X C D	
<code>next()</code>		current: X
<code>removeCurrent()</code>	playlist: A B C D	current: C
<code>prev()</code>		current: B
<code>setRepeat(true); next(); next(); next()</code>		current: A (wrapped from D)

Conditions

- Use your own circular doubly linked list with a `current` pointer.
- Keep the list physically circular. Repeat on or off should change only how `next` and `prev` behave, not the links.
- `toString()` prints the playlist from the first song, with the current one marked, for example A B [C] D.

Your Tasks

1. Draw the nodes and every pointer change for `playNext` and `removeCurrent`.
2. Implement all the operations and replay the example.
3. List every place where the *first* song (the head) must be updated, and test each one.
4. Add `shuffle()`, which plays every song exactly once in random order without changing the playlist. Which extra structure do you need?

Edge Cases to Consider

- Removing the only song
- Removing the current song when it is the last one and Repeat is off
- `prev()` on the first song with Repeat off
- `playNext` on an empty playlist

Check with AI (after you finish)

Ask an AI for a sequence of 10 operations most likely to break a circular doubly linked list. Run it, and check your list against your hand drawing.

Interview Angle

“Why doubly linked and not an array? Which operation would be $O(n)$ with an `ArrayList`?”

Question 10 - Banking: Adding Very Large Numbers

Topic: Linked list, recursion · Day 3, 4 · Level: Advanced

Scenario

A core-banking reconciliation job adds account balances that can be hundreds of digits long (amounts are stored in paise, and are aggregated across years and branches). These do not fit in `long`, and the team is not allowed to use `BigInteger`. Each number is stored as a singly linked list of digits, **most significant digit first**.

Example

```
Input:  7 -> 2 -> 4 -> 3      (7243)
        5 -> 6 -> 4            (564)
Output: 7 -> 8 -> 0 -> 7      (7807)

Input:  9 -> 9 -> 9   +   1
Output: 1 -> 0 -> 0 -> 0
```

Conditions

- The lists can have different lengths.
- Do not convert the lists to a number or a `String`.
- Solve it in **three ways**: (a) reverse both lists, add, and reverse the result; (b) without modifying the input lists, using your own stack; (c) using recursion, without modifying the input lists.

Your Tasks

1. Dry-run version (c) on the first example. Draw the call stack and show where each carry comes from.
2. Implement all three versions, and check that they produce identical output on 1,000 random pairs of up to 200 digits.
3. Compare the three versions' time and extra space in a table.

4. **Extension:** Implement subtraction $a - b$, where $a \geq b$. Remove leading zeros from the result.

Edge Cases to Consider

- A carry that creates a new leading digit ($999 + 1$)
- One number is 0
- Lists of very different lengths
- Leading zeros in the input ($0 \rightarrow 0 \rightarrow 4 + 5$)

Hint

For the recursive version, first pad the shorter list conceptually (or line the lists up using their lengths). The recursive call returns the carry from the digits to its right.

Interview Angle

"Your recursive solution crashed on a 100,000-digit number. Why? Which of your three versions would you ship to production?"

On the Job

Arbitrary-precision arithmetic in cryptography (RSA keys), financial ledgers and scientific computing.

Question 11 - Memory Allocator Free List

Topic: Sorted linked list, merging adjacent nodes · Day 3 · Level: Advanced

Scenario

Inside every runtime (the JVM, a game engine, an embedded OS) is a memory allocator. A simple design keeps a **free list**: a linked list of unused memory blocks, sorted by start address. `allocate(size)` finds the first free block large enough (*first fit*) and carves the request out of it. `free(id)` returns a block to the list, and **merges it with free neighbours** so that memory doesn't fragment.

Example

```
Heap of 100 bytes: free list = [0..99]

allocate(30)  -> A at 0..29      free: [30..99]
allocate(20)  -> B at 30..49     free: [50..99]
allocate(10)  -> C at 50..59     free: [60..99]
free(B)       free: [30..49] [60..99]
allocate(15)  -> D at 30..44     free: [45..49] [60..99]
free(C)       C is 50..59: joins 45..49 on its left and 60..99 on its right
              free: [45..99]    (one block of 55 bytes)
```

Conditions

- Free blocks are nodes in your own singly linked list, kept sorted by start address.
- Allocated blocks are tracked by id. A `HashMap<String, Block>` is allowed for this.
- `allocate` returns null (or throws) when no single free block is large enough, *even if the total free memory is enough*.

Your Tasks

1. Draw the free list after each step of the example.
2. Implement `allocate`, `free`, and `printFreeList()`.
3. List all four cases of `free`: no merge, merge left, merge right, and merge both. Write one test for each.
4. Construct a sequence in which total free memory is 50 bytes but `allocate(20)` still fails. This is called *external fragmentation*.
5. Implement *best fit* (the smallest block that fits) as an option, and compare fragmentation on a random workload.

Edge Cases to Consider

- Freeing a block at address 0, or at the end of the heap
- A request exactly the size of a free block (the node must disappear)
- Freeing the same id twice
- `allocate(0)` or a negative size

Think Before Coding

Because the list is sorted, a freed block's only possible neighbours are the free node just before it and the free node just after it. Find both in one pass.

Interview Angle

“What is the time complexity of `allocate` and `free` here? Real allocators use segregated free lists, one list per size class. Why?”

Question 12 - Disk Usage Analyzer

Topic: Recursion on an N-ary tree · Day 4, 5 · Level: Intermediate

Scenario

A developer's laptop is out of disk space. You are building a small tool, like `du` on Linux or WinDirStat, that models the file system as a tree and answers questions about it. Folders have any number of children, and files are leaves with a size.

Example

```
project/          30 KB
├── src/           25 KB
│   ├── Main.java 12 KB
│   ├── Utils.java 8 KB
│   └── model/      5 KB
│       └── User.java 5 KB
├── docs/          3 KB
│   └── README.md  3 KB
└── build.gradle   2 KB

find("User.java") -> project/src/model/User.java
```

Conditions

- Create the tree in code with your own `FsNode` class (name, size, list of children). A `java.util` list is allowed for the children.
- Every operation below must be **recursive**.
- Folder sizes are computed, never stored by hand.

Your Tasks

1. Print the tree with indentation and computed sizes, exactly like the example.
2. `totalSize(node)`: decide which traversal order (pre or post) this needs, and why.
3. `find(name)`: return the full path, or report that the file was not found.
4. `largestFiles(k)`: list the `k` largest files anywhere in the tree.
5. `depth()` and `countFiles()`, plus a list of the folders that are empty.
6. For each method, state its complexity in terms of `N` (the number of nodes) and `H` (the height).

Edge Cases to Consider

- An empty folder
- A file directly at the root
- Two files with the same name in different folders (return all paths)
- A very deep folder chain (what limits recursion depth?)

Hint

For `find`, pass the “path so far” down as a parameter, the same way you passed the “answer so far” in subset generation.

Interview Angle

“Real file systems have symbolic links, which can create cycles. How would your recursion loop forever, and how would you prevent it?”

On the Job

Disk analyzers, build tools that walk source trees, and cloud storage billing all work like this.

Question 13 - Hackathon Team Builder

Topic: Recursion, backtracking with pruning · Day 4 · Level: Advanced

Scenario

A company is forming teams for an internal hackathon. Each employee has a cost (the hours their manager can release). HR wants **every possible team of exactly k employees whose total cost does not exceed the budget**, so that they can pick the most balanced one.

Example

Employees: A = 4, B = 3, C = 5, D = 2, E = 6
k = 3, budget = 10

Valid teams:

A B D (cost 9)

B C D (cost 10)

Total valid teams: 2 (out of 10 possible teams of 3)

Conditions

- Generate teams by recursion (include or exclude each employee). Do not write nested loops.
- Each team is listed once, with members in input order.
- **Prune**: stop exploring a branch as soon as it can no longer produce a valid team.

Your Tasks

1. Draw the include/exclude recursion tree for the first three employees.
2. Implement the generator without pruning, and count the recursive calls.
3. Add two pruning rules: (a) the cost already exceeds the budget; (b) there aren't enough employees left to reach k . Count the calls again.
4. Run both versions on 25 random employees with $k = 5$ and compare the call counts.
5. **Extension**: Two employees, given as a pair, refuse to be on the same team. Add this constraint.

Edge Cases to Consider

- $k = 0$, $k = n$, $k > n$
- No valid team at all
- Employees with cost 0
- Duplicate costs

Think Before Coding

At each employee you make one decision: in or out. When can you be sure that *no* decision further down this path will work?

Interview Angle

"Does sorting the employees by cost first allow even stronger pruning? Which rule becomes possible?"

Check with AI (after you finish)

After both versions work, ask an AI to explain why your pruning is safe (that it never skips a valid team). Check its argument against your own recursion tree.

Question 14 - E-Commerce Price Filter

Topic: BST range queries, floor/ceiling, skewed trees · Day 5 · Level: Advanced

Scenario

An online store's product listing page has a price slider: *"Show products between Rs. 500 and Rs. 1500, cheapest first."* It also suggests *"the closest product under your budget"*. The catalogue team stores prices in a **binary search tree**.

Example

Insert prices in this order: 1200, 450, 2500, 300, 800, 1800, 3200, 650, 999

```
range(500, 1500) -> 650, 800, 999, 1200
floor(1000)      -> 999    (largest price <= 1000)
ceiling(2000)    -> 2500   (smallest price >= 2000)
```

Conditions

- Use your own BST. Each node holds a price and a list of product names (several products can share a price).
- `range(lo, hi)` must **skip** subtrees that cannot contain answers. Do not traverse the whole tree.
- `floor` and `ceiling` must run in $O(h)$ time.

Your Tasks

1. Draw the BST for the example insert order.
2. Implement `insert`, `range`, `floor` and `ceiling`. Count the nodes visited by `range(500, 1500)` and compare with 9.
3. **Production bug:** The catalogue is bulk-loaded from a CSV file that is already sorted by price. Insert 1 to 10,000 in sorted order and print the tree height. Then insert the same values in random order and compare.
4. With 100,000 sorted inserts, a recursive `insert` crashes. Explain why, and write an iterative `insert`.
5. Suggest an order in which to insert a sorted list so that the tree stays balanced. Hint: think of binary search.

Edge Cases to Consider

- A range with $lo > hi$
- An empty range
- `floor` smaller than every price (no answer)
- Duplicate prices
- An empty tree

Think Before Coding

At a node whose price is below `lo`, can anything in its *left* subtree be in range?

Interview Angle

"Java's `TreeMap` gives `floorKey`, `ceilingKey` and `subMap` in $O(\log n)$ even for sorted input. What does it do differently from your BST?" (This is a preview of Day 6.)

On the Job

Price filters, time-range queries on logs, leaderboards and auto-complete all rely on ordered search trees.

Question 15 - Company Org Chart

Topic: General tree, BFS, common ancestor · Day 5 · Level: Advanced

Scenario

An HR system stores employees as flat records exported from a database: each record has an employee id, a name and a manager id. The product team wants an org-chart feature built from these records.

Example

id	name	managerId
1	Asha	0 (CEO)
2	Ravi	1
3	Meera	1
4	Karan	2
5	Neha	2
6	Arjun	3
7	Pooja	4

Levels: L0: Asha | L1: Ravi Meera | L2: Karan Neha Arjun | L3: Pooja
Chain of Pooja: Pooja -> Karan -> Ravi -> Asha
Common manager: (Pooja, Neha) -> Ravi (Pooja, Arjun) -> Asha
Total reports of Ravi (direct + indirect): 3

Conditions

- Build the tree from records that may arrive in **any order**, for example a child before its manager.
- A `HashMap<Integer, Employee>` is allowed for lookup. The level-wise print must use your own queue.
- Each feature below should visit only the nodes it needs.

Your Tasks

1. Build the tree, and print it level by level.
2. `chain(id)`: the reporting chain up to the CEO.
3. `commonManager(a, b)`: the lowest common manager. Do it in two ways: using parent pointers, and using recursion from the root.
4. `totalReports(id)` for every employee, in a **single** traversal. Which traversal order makes this possible?
5. Validate the data and report: more than one CEO, a manager id that doesn't exist, and a **cycle** (A reports to B, and B reports to A).

Edge Cases to Consider

- One employee is the other's manager (the common manager is that employee)
- The same employee passed twice
- The CEO's own chain
- A company with only the CEO

Hint

For the parent-pointer version, record the depth of both employees, lift the deeper one until both are at the same level, then move both up together.

Interview Angle

"The org has 3 lakh employees and the common-manager query runs millions of times a day. How would you speed it up?" Discuss precomputing depths, and caching each employee's chain.

On the Job

HRMS tools such as Workday and SAP SuccessFactors, approval workflows ("route to the common manager") and access-control hierarchies.

Question 16 - Saving a Tree to Disk or Over the Network

Topic: Tree traversal, recursion, serialization · Day 4, 5 · Level: Advanced

Scenario

A decision-tree model for loan approval is built on a server and must be **sent to mobile apps**, and also **cached in Redis** as a string. Objects with pointers can't be sent directly, so the tree must be converted to a string (*serialization*) and rebuilt exactly on the other side (*deserialization*).

Example

Tree:

```
      1
     / \
    2   3
     / \
    4   5
```

`serialize(root)` -> "1,2,#,#,3,4,#,#,5,#,#" (preorder, # = null)
`deserialize(...)` -> the same tree (same shape, same values)

Conditions

- Work on a binary tree of integers (values can be negative or have several digits).
- `deserialize(serialize(t))` must reproduce the exact same shape.
- Both directions must be $O(n)$.

Your Tasks

1. Explain why preorder *without* null markers (1,2,3,4,5) is not enough. Draw two different trees that give the same output.
2. Implement `serialize` and `deserialize` recursively, and dry-run `deserialize` on the example string.
3. Write `isSameTree(a, b)` and use it to verify the round trip on 1,000 randomly generated trees.
4. Implement a second format based on level order (like LeetCode's [1,2,3,null,null,4,5]) using your own queue.

5. **For a BST**, show that the preorder alone (no null markers) is enough to rebuild it. Implement this, and explain why the trick fails for a general binary tree.

Edge Cases to Consider

- An empty tree
- A single node
- Negative and multi-digit values
- A completely skewed tree of 10,000 nodes (does your recursion survive?)

Think Before Coding

During deserialization, how does the function know when a subtree is finished? What does each # tell it?

Interview Angle

"Your string is too large to send. Suggest two ways to make it smaller, and the trade-off of each."

On the Job

Saving ML models (decision trees, random forests), caching DOM trees, and syncing JSON documents between client and server.

Check with AI (after you finish)

After your round-trip test passes, ask an AI for inputs that might break your parser (spaces, a trailing comma, malformed strings). Decide which ones your API should reject.